

文档解析改

文档解析模块设计文档

项目定位: Python RAG 执行端前置模块，负责多格式非结构化文档的精准解析与标准 Markdown 格式转换。

协作模式: Java 管理端 ←(共用数据库 / MQ)→ Python 执行端

阶段一：需求深度分析

1.1 核心业务逻辑

文档解析模块旨在提供统一的文档处理抽象层，支持无缝接入和解析不同格式的底层文件（如 PDF、Word、HTML、TXT 等），屏蔽底层的解析复杂度和差异性，最终向后道工序输出高质量、结构化的 Markdown 纯文本数据。

1.2 架构定位

Plaintext



1.3 边界与异常场景

场景	潜在问题	处理策略
超大 PDF 文件处理	OOM (内存溢出) 或单次解析耗时过长	异步任务队列处理，流式按页读取解析，避免一次性加载入内存
纯图片/扫描版 PDF	常规物理提取失败，返回空白	前置检测，自动路由至 OCR (光学字符识别) 视觉大模型辅助解析

场景	潜在问题	处理策略
HTML 包含大量噪音	解析出大量广告、脚本等无用字符	引入基于文本密度的预处理清洗算法，精确定位正文 DOM 节点
文件损坏或不支持格式	解析引擎报错阻断进程	快速失败 (Fail-fast)，捕获异常并向 Java 端回写 ParseFailedException

1.4 非功能性需求

- 并发支持:** 能够通过异步 Worker (如 Celery) 水平扩展，支持高并发大文件批量解析。
- 解析精度:** 核心格式样式映射准确率高（如 Word 的标题等级、加粗列表需完美映射为 Markdown）。
- 高扩展性:** 新增文档格式解析器无需修改核心调度代码 (符合开闭原则)。
- 容错机制:** 批处理任务中单个文档解析失败不可导致整体 Worker 崩溃，需具备强隔离性。

阶段二：数据模型与存储设计

2.1 核心实体关系

Plaintext

DocumentRecord (Java端维护的文档元数据)	DocumentParseTask (解析任务执行记录)
id (PK)	id (PK)
user_id	document_id (FK) ----->
DocumentRecord	
file_name 1:1	status
(pending/processing/success/failed)	
file_url (OSS路径)	markdown_content (LONGTEXT)
file_type (pdf/docx/html)	error_message

2.2 数据库表结构设计

2.2.1 文档解析任务表 (document_parse_task)

字段名	数据类型	主键	索引	必填	注释
id	VARCHAR(36)	✓		✓	任务唯一标识 (UUID)
document_id	VARCHAR(36)		✓	✓	关联的原始文档记录 ID

字段名	数据类型	主键	索引	必填	注释
file_type	VARCHAR(32)			✓	文件真实格式 (pdf, docx, html)
status	VARCHAR(16)		✓	✓	状态: pending/processing/success/fai
markdown_content	LONGTEXT				解析成功后的标准 Markdown 文
page_count	INT				解析出的总页数/大致长度指标
error_message	VARCHAR(512)				失败时的具体堆栈或异常描述
time_cost_ms	INT				解析耗时 (毫秒)
created_at	DATETIME			✓	任务创建时间
updated_at	DATETIME			✓	任务更新时间

2.3 Redis 缓存与队列策略

由于解析任务通常属于长耗时 I/O 与 CPU 密集型操作，建议引入 Redis 作为任务队列和状态缓存。

Python

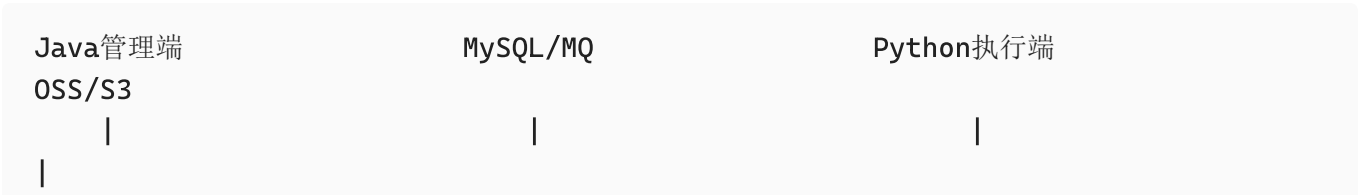
```
# Redis Key 设计
Key: parse:task:queue           -> List (存放待处理的任务 ID, 供异步 Worker
消费)
Key: parse:task:{task_id}:status -> String (缓存解析进度或实时状态, 供前端短
轮询)
Key: parse:lock:{document_id}   -> String (分布式锁, 防止同一文件被并发重复
解析)

# 淘汰策略
针对 status 缓存, 设置 TTL: 3600s (1小时), 落库后即可失效。
```

阶段三：架构与模块设计

3.1 系统流转时序图

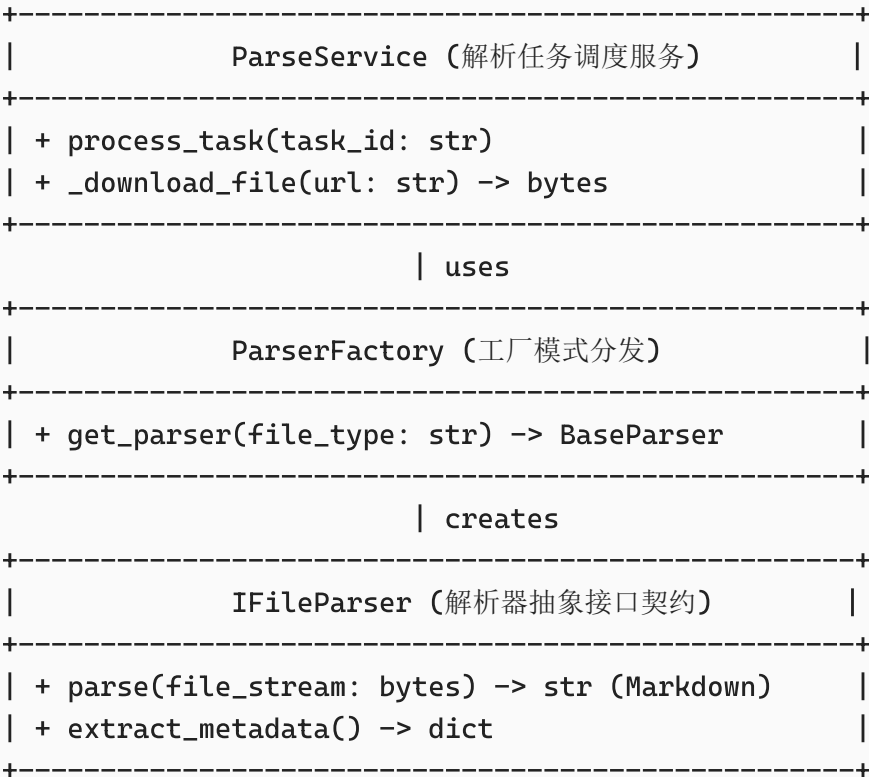
Plaintext

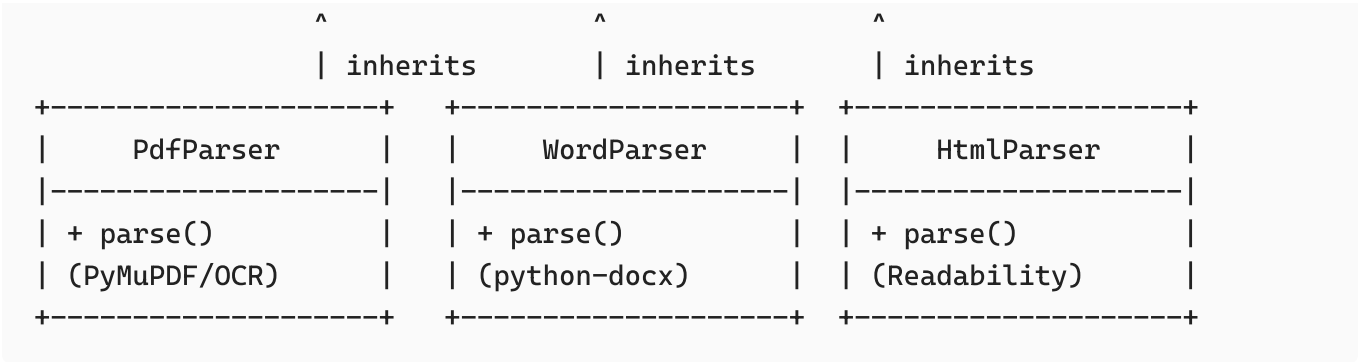




3.2 核心类图抽象

Plaintext





3.3 关键技术栈决策

决策点	方案选型	核心理由
抽象层	Python ABC	明确文件解析契约，强制子类实现 <code>parse</code> 返回 Markdown 方法。
设计模式	简单工厂模式	通过文件扩展名或 MIME Type 动态分发给不同的解析引擎，便于后续新增 PPT、Excel 解析器。
Word 解析	python-docx / py pandoc	能精准遍历 <code>word/document.xml</code> 树，将内建样式映射为 # 标题和无序列表。
HTML 解析	Readability-lxml / Trafilatura	业界标杆，利用文本密度评分启发式剔除广告和导航条，精准提纯正文。
PDF 解析	PyMuPDF (fitz) + RapidOCR	PyMuPDF 获取排版坐标；对于扫描件，自动 fallback 调用本地轻量 OCR 提取文本。
异步调度	Celery + Redis	解析大文件耗时较长，避免 HTTP 阻塞，保障 Python 服务的高吞吐量。

阶段四：接口规约文档 (Python API)

4.1 核心解析接口 (同步直接返回版 - 适用于小文件测试)

- `POST /api/v1/parser/extract_sync`
- 接口名称: 同步提取文档转 Markdown

请求参数 (Multipart/form-data):

字段名	类型	必填	描述
file	Binary	✓	上传的二进制文件流
file_type	string	✓	文档格式，例如 pdf , docx , html

字段名	类型	必填	描述
use_ocr	boolean	X	是否强制开启 OCR 解析扫描件 (默认 false)

响应示例 (JSON):

JSON

```
{
  "code": 200,
  "message": "success",
  "data": {
    "file_type": "docx",
    "markdown": "# 需求深度分析\n\n多LLM接入模块旨在提供...\n\n## 架构定位\nJava 管理端...",
    "metadata": {
      "pages_or_length": 5,
      "time_cost_ms": 320
    }
  }
}
```

4.2 异步解析任务提交接口 (工业级调用)

- **POST** /api/v1/parser/task/submit
- **接口名称:** 提交异步解析任务

请求参数 (JSON):

字段名	类型	必填	描述
task_id	string	✓	Java 端生成的解析任务 ID
document_id	string	✓	文档 ID
file_url	string	✓	内部 OSS/S3 下载直链
file_type	string	✓	文档后缀

响应示例 (JSON):

JSON

```
{
  "code": 200,
  "message": "Task accepted and queued",
}
```

```
"data": {
  "task_id": "task_uuid_12345",
  "status": "pending"
}
```

阶段五：工程目录结构规划

(注：根据要求，本目录结构去除了所有分片 Chunking 相关逻辑，仅保留解析至 Markdown 的能力)

Plaintext

```
src/
├── core/
│   ├── parser/
│   │   ├── __init__.py
│   │   ├── interfaces.py
│   │   ├── base_parser.py
│   │   ├── factory.py
│   │   ├── providers/
│   │   │   ├── __init__.py
│   │   │   ├── word_parser.py
│   │   │   ├── html_parser.py
│   │   │   ├── pdf_parser.py
│   │   │   └── txt_parser.py
│   │   ├── utils/
│   │   │   ├── text_formatter.py
│   │   │   └── file_downloader.py
│   │   └── exceptions.py
│   └── ParseTimeout()
├── services/
│   ├── parse_task_service.py
│   └── mq_consumer.py
├── api/
│   ├── routes/
│   │   └── parse.py
├── models/
│   └── parse_task.py
```

IFileParser 抽象接口契约（必须返回 **Markdown**）
BaseParser 解析器基类，包含通用的文件校验逻辑
ParserFactory 格式分发工厂
具体的解析引擎实现目录

docx -> Markdown 转换逻辑
网页去噪提取正文逻辑
PDF 坐标提取与排版还原逻辑
纯文本标准化读取

Markdown 文本统一清洗/排版工具
OSS 流式下载工具类
自定义解析异常（如 **UnsupportedFormat**,

核心服务：接收 **task** -> 下载 -> 解析 -> 落库
Celery 或 **RabbitMQ** 的异步任务消费者

对外暴露的 **HTTP** 解析触发与查询接口

SQLAlchemy ORM: **document_parse_task** 表定义

阶段六：Java 端与 Python 端协同边界总结

为了保证系统架构的清晰，团队在研发文档解析模块时需严格遵循以下数据边界：

数据模块操作	Java 管理端权限	Python 执行端权限	通信方式
原始文档上传与 OSS 存储	核心执行	只读下载流	HTTP OSS API
解析任务初始化 (Pending)	写入主导	不介入	MySQL 直连
解析任务下发与触发	生产者 (发消息)	消费者 (接消息)	Redis/RabbitMQ
文档提取与 Markdown 转换	不介入	核心执行	内存计算密集
解析结果 (Markdown) 持久化	不介入	写入主导	MySQL 直连回写
解析状态展示与管理面板	读取并呈现	不介入	查询 MySQL